

SCARA Robot — Giải Thích Chi Tiết Code

1. Thư Viện & Định Nghĩa (Dòng 1–23)

```
#include <AccelStepper.h> // Thư viện điều khiển stepper có gia tốc
#include <math.h>          // Hàm toán: sin, cos, atan2, acos, sqrt

#define limitSwitch1 9     // Công tắc hành trình Joint 1 → chân D9
#define limitSwitch2 10    // Công tắc hành trình Joint 2 → chân D10
#define EN_PIN 8           // Chân Enable chung CNC Shield → D8
```

- **AccelStepper**: thư viện cho phép chạy stepper với profile tăng/giảm tốc (trapezoidal). Tham số `1` = chế độ DRIVER (chỉ dùng STEP + DIR).
- **CNC Shield V3**: shield cắm lên Arduino UNO, có 3 kênh X/Y/Z. Chân STEP và DIR được nối cố định.

```
AccelStepper stepper1(1, 2, 5); // Joint 1: STEP=D2, DIR=D5 (kênh X)
AccelStepper stepper2(1, 3, 6); // Joint 2: STEP=D3, DIR=D6 (kênh Y)
AccelStepper stepperZ(1, 12, 13); // Trục Z: STEP=D12, DIR=D13 (custom)
```

Trục Z không dùng kênh Z mặc định (D4/D7) mà nối tới D12/D13, có thể do layout PCB hoặc tránh xung đột.

Hằng số cơ khí

```
const float L1 = 228.0;           // Link 1 = 228mm
const float L2 = 136.5;           // Link 2 = 136.5mm
const float theta1AngleToSteps = 44.444444; // 1° Joint1 = 44.44 bước
const float theta2AngleToSteps = 35.555555; // 1° Joint2 = 35.56 bước
const float zMmToSteps = 400.0;   // 1mm trục Z = 400 bước
const float HOMING_BACKOFF_ANGLE_X = 5.0; // Lùi 5° sau khi chạm SW1
const float HOMING_BACKOFF_ANGLE_Y = 150.0; // Lùi 150° sau khi chạm SW2
```

- `theta1AngleToSteps` : tính từ tỷ số truyền (pulley/gear ratio) × số bước/vòng × microstepping.
Ví dụ: Motor 200 step/rev × 16 microstep × gear ratio / 360° = 44.44
- `HOMING_BACKOFF_ANGLE_Y = 150°` : Joint 2 lùi rất xa vì limit switch đặt ở vị trí cực hạn, cần quay nhiều để về vị trí "0° thực tế".

2. Biến Trạng Thái (Dòng 25–31)

```

bool isHomed = false;      // Cờ: đã homing chưa?
float currentX = 0.0;      // Tọa độ X hiện tại (mm)
float currentY = 0.0;      // Tọa độ Y hiện tại (mm)
float currentAngle1 = 0.0; // Góc Joint 1 hiện tại (độ)
float currentAngle2 = 0.0; // Góc Joint 2 hiện tại (độ)
float currentZ = 0.0;      // Chiều cao Z hiện tại (mm)

```

Các biến này được cập nhật sau **mỗi lần di chuyển** để code luôn biết vị trí robot, phục vụ cho:

- Tính nội suy (cần biết điểm xuất phát)
- Gửi báo cáo `STATUS` về GUI

3. Hàm `setup()` (Dòng 33–46)

```

void setup() {
  Serial.begin(115200);      // Baud rate cao cho giao tiếp nhanh
  pinMode(limitSwitch1, INPUT_PULLUP); // Kích hoạt điện trở kéo lên nội
  pinMode(limitSwitch2, INPUT_PULLUP); // → Bình thường đọc HIGH
  pinMode(EN_PIN, OUTPUT);
  digitalWrite(EN_PIN, LOW); // LOW = bật driver (active-low)

  stepper1.setMaxSpeed(2000); stepper1.setAcceleration(1000);
  stepper2.setMaxSpeed(2000); stepper2.setAcceleration(1000);
  stepperZ.setMaxSpeed(4000); stepperZ.setAcceleration(4000);
}

```

- **INPUT_PULLUP**: limit switch nối giữa chân tín hiệu và GND. Khi chưa chạm: HIGH. Khi chạm: LOW.
- **EN_PIN LOW**: CNC Shield dùng logic active-low cho enable.
- Trục Z có tốc độ/gia tốc gấp đôi vì thường cần di chuyển nhanh (nhắc/hạ bút).

4. Hàm `reportStatus()` (Dòng 48–57)

```

void reportStatus() {
  Serial.print("STATUS,");
  Serial.print(currentX); Serial.print(",");
  Serial.print(currentY); Serial.print(",");
  Serial.print(currentZ); Serial.print(",");
  Serial.print(currentAngle1); Serial.print(",");
  Serial.print(currentAngle2);
}

```

```
Serial.println();  
}
```

Gửi chuỗi dạng CSV: `STATUS,X,Y,Z,01,02\n`

Python GUI dùng `split(",")` để parse. Prefix `STATUS` giúp phân biệt với các dòng log/debug khác.

5. Hàm `loop()` — Command Parser (Dòng 59–177)

Cấu trúc tổng quát

```
void loop() {  
    if (Serial.available() > 0) {  
        String input = Serial.readStringUntil('\n'); // Đọc 1 dòng  
        input.trim();                               // Xóa \r, space thừa  
          
        if (input.length() > 0) {  
            char firstChar = input.charAt(0);       // Lấy ký tự lệnh  
            // ... phân nhánh xử lý ...  
            reportStatus();                         // Luôn báo cáo sau lệnh  
        }  
    }  
}
```

Lệnh H — Homing (dòng 68–70)

```
if (firstChar == 'H' || firstChar == 'h') {  
    homing(); // Gọi hàm homing, không cần tham số  
}
```

Đây là lệnh duy nhất **không cần** `isHomed == true`.

Gate kiểm tra Homing (dòng 71–75)

```
else {  
    if (!isHomed) {  
        Serial.println("\n[!] LOI: Ban phai go 'H' de xet goc truoc.");  
        return; // Thoát loop() ngay, không xử lý tiếp  
    }  
    // ... các lệnh khác ...  
}
```

Lệnh J — Jog (dòng 78–93)

```
// Format: J,<jointId>,<deltaAngle>
// VD: J,1,5.0 → Joint 1 quay thêm 5°
int jointId = input.substring(comma1+1, comma2).toInt();
float value = input.substring(comma2+1).toFloat();

if (jointId == 1) moveToAngleWithZ(currentAngle1 + value, currentAngle2, currentZ);
if (jointId == 2) moveToAngleWithZ(currentAngle1, currentAngle2 + value, currentZ);
if (jointId == 3) moveToAngleWithZ(currentAngle1, currentAngle2, currentZ + value);
```

- Jog = di chuyển **tương đối** (cộng thêm `value` vào góc hiện tại)
- Joint 3 thực ra là trục Z (tịnh tiến, không phải quay)

Lệnh A — Góc Tuyệt Đối (dòng 96–106)

```
// Format: A,<θ1>,<θ2>,<Z>
// VD: A,45,90,50 → Joint1=45°, Joint2=90°, Z=50mm
float t1 = input.substring(c1+1, c2).toFloat();
float t2 = input.substring(c2+1, c3).toFloat();
float z = input.substring(c3+1).toFloat();
moveToAngleWithZ(t1, t2, z);
```

Dùng **Động học thuận**: người dùng chỉ định trực tiếp góc từng khớp.

Lệnh P — Tọa Độ XY (dòng 108–119)

```
// Format: P,<X>,<Y>
// VD: P,200,150 → Di chuyển end-effector tới (200, 150)
float targetX = input.substring(c1+1, c2).toFloat();
float targetY = input.substring(c2+1).toFloat();
moveToXYZ(targetX, targetY, currentZ); // Giữ nguyên Z
```

Dùng **Động học ngược**: tính θ_1, θ_2 từ tọa độ Cartesian.

Lệnh D — Vẽ Đường Thẳng (dòng 121–144)

```
// Format: D,<x1>,<y1>,<x2>,<y2>,<zUp>,<zDown>
moveToXYZ(currentX, currentY, zUp); // 1. Nhắc bút
moveToXYZ(x1, y1, zUp); // 2. Bay tới điểm đầu
moveToXYZ(x1, y1, zDown); // 3. Hạ bút
drawLine(x2, y2); // 4. Vẽ nội suy
moveToXYZ(currentX, currentY, zUp); // 5. Nhắc bút
```

Quy trình 5 bước đảm bảo bút không kéo vạch khi di chuyển tới điểm bắt đầu.

Lệnh C — Vẽ Hình Tròn (dòng 146–170)

```
// Format: C,<cx>,<cy>,<r>,<zUp>,<zDown>
float startX = cx + r; // Điểm bắt đầu: mép phải (góc 0°)
float startY = cy;

moveToXYZ(currentX, currentY, zUp); // 1. Nhắc bút
moveToXYZ(startX, startY, zUp); // 2. Bay tới mép tròn
moveToXYZ(startX, startY, zDown); // 3. Hạ bút
drawCircle(cx, cy, r); // 4. Vẽ vòng tròn
moveToXYZ(currentX, currentY, zUp); // 5. Nhắc bút
```

6. Hàm `homing()` (Dòng 182–241)

Bước 1: Đọc trạng thái bình thường

```
int normalState1 = digitalRead(limitSwitch1); // Ghi nhớ trạng thái chưa chạm
int normalState2 = digitalRead(limitSwitch2);
```

So sánh với trạng thái này để phát hiện khi switch **thay đổi** (chạm). Cách này không phụ thuộc vào việc switch là NC hay NO.

Bước 2: Chạy về gốc đồng thời

```
stepper1.setSpeed(-1000); // Tốc độ âm = quay ngược chiều
stepper2.setSpeed(-1000);

while (!isJ1Homed || !isJ2Homed) {
  if (!isJ1Homed) {
    if (digitalRead(limitSwitch1) == normalState1)
      stepper1.runSpeed(); // runSpeed() = chạy tốc độ không đổi, không gia tốc
    else
      isJ1Homed = true; // Chạm switch → dừng joint này
  }
  // Tương tự cho Joint 2...
}
```

- `runSpeed()` : chạy ở tốc độ cố định (không dùng profile gia tốc)
- Mỗi joint dừng **độc lập** khi chạm switch riêng của nó

Bước 3: Lùi ra (Backoff)

```
long backoffSteps1 = HOMING_BACKOFF_ANGLE_X * theta1AngleToSteps; // 5° × 44.44 ≈ 222 bước
long backoffSteps2 = HOMING_BACKOFF_ANGLE_Y * theta2AngleToSteps; // 150° × 35.56 ≈ 5333 bước
```

```
stepper1.move(backoffSteps1); // move() = di chuyển tương đối
stepper2.move(backoffSteps2);

while (stepper1.distanceToGo() != 0 || stepper2.distanceToGo() != 0) {
    stepper1.run(); // run() = chạy có gia tốc
    stepper2.run();
}
```

- `move()` vs `moveTo()` : `move` là tương đối, `moveTo` là tuyệt đối
- Lùi đồng thời cả 2 joint, kết thúc khi cả 2 về xong

Bước 4: Thiết lập gốc

```
stepper1.setCurrentPosition(0); // Vị trí hiện tại = 0 bước
stepper2.setCurrentPosition(0);
stepperZ.setCurrentPosition(0);

currentAngle1 = 0; currentAngle2 = 0; currentZ = 0;
currentX = L1 + L2; // = 228 + 136.5 = 364.5mm
currentY = 0.0;
isHomed = true;
```

Khi $\theta_1=0$, $\theta_2=0$: cánh tay duỗi thẳng theo trục X $\rightarrow X = L_1+L_2$, $Y = 0$.

7. Hàm `moveToAngleWithZ()` (Dòng 243–263)

```
void moveToAngleWithZ(float theta1_deg, float theta2_deg, float z_mm) {
    // Chuyển góc  $\rightarrow$  vị trí bước tuyệt đối
    stepper1.moveTo(theta1_deg * theta1AngleToSteps);
    stepper2.moveTo(theta2_deg * theta2AngleToSteps);
    stepperZ.moveTo(z_mm * zMmToSteps);

    // Blocking: chờ cả 3 trục về đích
    while (stepper1.distanceToGo() != 0 || stepper2.distanceToGo() != 0
        || stepperZ.distanceToGo() != 0) {
        stepper1.run();
        stepper2.run();
        stepperZ.run();
    }
}
```

- `moveTo()` : đặt vị trí đích tuyệt đối (tính bằng bước)
- 3 trục chạy **đồng thời** trong cùng vòng while, AccelStepper tự tính profile gia tốc cho từng trục

- Hàm **blocking**: không trả về cho đến khi tất cả trục đã tới đích

```
// Cập nhật FK
currentAngle1 = theta1_deg;
currentAngle2 = theta2_deg;
currentZ = z_mm;

float t1_rad = theta1_deg * PI / 180.0;
float t2_rad = theta2_deg * PI / 180.0;
currentX = L1 * cos(t1_rad) + L2 * cos(t1_rad + t2_rad);
currentY = L1 * sin(t1_rad) + L2 * sin(t1_rad + t2_rad);
}
```

Sau khi di chuyển, tính toán vị trí Cartesian (XY) bằng **Forward Kinematics** để đồng bộ biên trạng thái.

8. Hàm `moveToXYZ()` (Dòng 265–283)

Kiểm tra workspace

```
float r_square = (x*x) + (y*y);
if (r_square > pow(L1+L2, 2) || r_square < pow(L1-L2, 2)) return;
```

- $r^2 > (L1+L2)^2$: điểm quá xa, cánh tay không vươn tới
- $r^2 < (L1-L2)^2$: điểm quá gần, nằm trong "lỗ hổng" workspace
- Nếu ngoài workspace → **return ngay**, không báo lỗi (silent fail)

Tính Inverse Kinematics

```
// Bước 1: Tính  $\theta_2$ 
float cosTheta2 = (r_square - L1*L1 - L2*L2) / (2*L1*L2);
cosTheta2 = constrain(cosTheta2, -1.0, 1.0); // Clamp tránh lỗi acos
float theta2_rad = acos(cosTheta2); // Luôn trả về  $[0, \pi]$  → elbow-up

// Bước 2: Tính  $\theta_1$ 
float term1 = atan2(y, x); // Góc tới điểm đích
float term2 = atan2(L2*sin(theta2_rad), L1 + L2*cos(theta2_rad)); // Bù offset do khuỷu
float theta1_rad = term1 - term2;
```

Công thức IK cho robot 2R phẳng:

- $\theta_2 = \text{acos}(\dots)$ → chỉ có 1 nghiệm (elbow-up)
- $\theta_1 = \text{atan2}(y, x) - \text{atan2}(\dots)$ → tính góc vai có tính đến ảnh hưởng của khuỷu

```
moveToAngleWithZ(theta1_deg, theta2_deg, z); // Gọi hàm di chuyển
currentX = x; currentY = y; // Ghi đè XY = giá trị đích (chính xác hơn FK)
```

`currentX/Y` được gán trực tiếp thay vì dùng kết quả FK từ `moveToAngleWithZ()` để tránh tích lũy sai số số học.

9. Hàm `drawLine()` (Dòng 288–309)

```
void drawLine(float targetX, float targetY) {
    float dx = targetX - currentX;
    float dy = targetY - currentY;
    float distance = sqrt(dx*dx + dy*dy);

    float stepSize = 0.3; // Mỗi bước nội suy = 0.3mm
    int numSegments = max(1, (int)(distance / stepSize));
```

- Chia đường thẳng thành nhiều đoạn nhỏ 0.3mm
- Ví dụ: đường 30mm → 100 đoạn → 100 lần gọi IK

```
// Đổi sang chế độ vẽ: chậm + gia tốc cực cao
stepper1.setAcceleration(50000); stepper2.setAcceleration(50000);
stepper1.setMaxSpeed(800); stepper2.setMaxSpeed(800);

for (int i = 1; i <= numSegments; i++) {
    float nextX = currentX + dx * ((float)i / numSegments); // Nội suy tuyến tính
    float nextY = currentY + dy * ((float)i / numSegments);
    calculateAndRunIK_Drawing(nextX, nextY);
}

// Khôi phục tốc độ bình thường
stepper1.setAcceleration(1000); stepper2.setAcceleration(1000);
stepper1.setMaxSpeed(2000); stepper2.setMaxSpeed(2000);
currentX = targetX; currentY = targetY;
}
```

- Gia tốc 50000: motor đạt tốc độ đích gần như **ngay lập tức** → không có giai đoạn tăng/giảm tốc giữa các đoạn nhỏ → chuyển động đều
- Tốc độ 800: chậm hơn bình thường để vẽ chính xác
- `currentX/Y` dùng giá trị **ban đầu** (trước vòng for) cho phép tính `nextX/Y`, tránh drift

10. Hàm `drawCircle()` (Dòng 311–328)

```
void drawCircle(float cx, float cy, float r) {  
    int numSegments = max(10, (int)(2*PI*r / 0.5)); // Bước cung ≈ 0.5mm
```

Số đoạn phụ thuộc chu vi: $C = 2\pi r$. Ví dụ $r=20\text{mm} \rightarrow C \approx 125.7\text{mm} \rightarrow 251$ đoạn.

```
    for (int i = 1; i <= numSegments; i++) {  
        float angle = 2.0 * PI * i / numSegments; // Chia đều 360°  
        float nextX = cx + r * cos(angle); // Tọa độ điểm trên vòng tròn  
        float nextY = cy + r * sin(angle);  
        calculateAndRunIK_Drawing(nextX, nextY);  
    }
```

- Bắt đầu từ `i=1` (bỏ qua điểm start vì robot đã ở đó)
- Kết thúc tại `i=numSegments` → `angle = 2π` → quay đủ 360°
- Điểm cuối trùng điểm đầu: `(cx+r, cy)` → vòng tròn khép kín

```
    currentX = cx + r; // Reset về vị trí (cx+r, cy) sau khi vẽ xong  
    currentY = cy;  
}
```

11. Hàm `calculateAndRunIK_Drawing()` (Dòng 330–355)

Đây là hàm IK **tối ưu cho vẽ**: chỉ điều khiển Joint 1 và Joint 2 (không Z).

```
void calculateAndRunIK_Drawing(float x, float y) {  
    // Kiểm tra workspace (giống moveToXYZ)  
    float r_square = (x*x) + (y*y);  
    if (r_square > pow(L1+L2,2) || r_square < pow(L1-L2,2)) return;  
  
    // Tính IK (giống moveToXYZ)  
    float cosTheta2 = (r_square - L1*L1 - L2*L2) / (2*L1*L2);  
    cosTheta2 = constrain(cosTheta2, -1.0, 1.0);  
    float theta2_rad = acos(cosTheta2);  
    float theta1_rad = atan2(y,x) - atan2(L2*sin(theta2_rad), L1+L2*cosTheta2);
```

Phần IK giống hệt `moveToXYZ()`, nhưng khác ở phần thực thi:

```
    // Di chuyển CHỈ 2 trục (không Z)  
    stepper1.moveTo(theta1_deg * theta1AngleToSteps);  
    stepper2.moveTo(theta2_deg * theta2AngleToSteps);
```

```

while (stepper1.distanceToGo() != 0 || stepper2.distanceToGo() != 0) {
  stepper1.run();
  stepper2.run();
}

// Cập nhật góc (QUAN TRỌNG)
currentAngle1 = theta1_deg;
currentAngle2 = theta2_deg;
}

```

Tại sao tách riêng thay vì dùng `moveToXYZ()` ?

1. **Không điều khiển Z:** khi vẽ, Z cố định → không cần gọi `stepperZ.run()` mỗi đoạn
2. **Không gọi `moveToAngleWithZ()`** : tránh overhead tính FK mỗi đoạn nhỏ
3. **Chỉ cập nhật `currentAngle`** : không cập nhật `currentX/Y` vì `drawLine()` / `drawCircle()` tự quản lý

Dòng comment cuối: "LƯU LẠI GÓC SAU KHI VẼ ĐỂ KHÔNG BỊ GIẬT LÙI TRỤC Z" — Nếu không cập nhật `currentAngle`, lần gọi `moveToAngleWithZ()` tiếp theo sẽ tính sai vị trí đích vì `currentAngle` cũ.

12. Bảng Tổng Hợp Các Hàm

Hàm	Dòng	Input	Output	Blocking?
<code>setup()</code>	33–46	—	Khởi tạo HW	—
<code>reportStatus()</code>	49–57	—	Serial <code>STATUS,...</code>	Không
<code>loop()</code>	59–177	Serial input	Gọi hàm tương ứng	—
<code>homing()</code>	182–241	—	Set isHomed=true	Có
<code>moveToAngleWithZ()</code>	243–263	θ_1, θ_2, Z	Chạy 3 motor + FK	Có
<code>moveToXYZ()</code>	265–283	X, Y, Z	IK → <code>moveToAngleWithZ</code>	Có
<code>drawLine()</code>	288–309	targetX, targetY	Nội suy + vẽ	Có
<code>drawCircle()</code>	311–328	cx, cy, r	Nội suy + vẽ	Có
<code>calculateAndRunIK_Drawing()</code>	330–355	X, Y	IK + chạy 2 motor	Có

13. Luồng Gọi Hàm

loop() đọc Serial

└ 'H' → homing() → reportStatus()

└ 'J' → moveToAngleWithZ() → reportStatus()

└ 'A' → moveToAngleWithZ() → reportStatus()

└ 'P' → moveToXYZ() → moveToAngleWithZ() → reportStatus()

└ 'D' → moveToXYZ() ×3 → drawLine() → calculateAndRunIK_Drawing() ×N → moveToXYZ() → reportStatus()

└ 'C' → moveToXYZ() ×3 → drawCircle() → calculateAndRunIK_Drawing() ×N → moveToXYZ() → reportStatus()